

Build an Enhanced MCP Server

Estimated time needed: 30 minutes

Ever wished your AI assistant could not only talk, but also read, write, and interact directly with your files? With the Model Context Protocol, you can widen an LLM's capabilities through tools, resources and prompts.

Model Context Protocol

[Large Language Models](#) (LLMs) are powerful, but on their own, they can't do much outside of text generation. That's where the **Model Context Protocol (MCP)** comes in. MCP is a standardized way to let models talk to their environment using **tools, resources, and prompts**.

How MCP Differs from Normal APIs

- **Standardization & modularity** → MCP provides a universal protocol that any LLM can use directly, instead of bespoke API integrations.
- **Three capability types** → Unlike APIs that just expose endpoints, MCP formalizes **tools, resources, and prompts** to cover actions, data, and workflows.
- **Contextual & interactive** → MCP servers run alongside the model, enabling real-time, two-way interaction (including eliciting missing info), not just one-way request/response.

Learning Objectives

By completing this lab you will be able to:

- Explain what the **Model Context Protocol** is and why it matters for enabling LLMs to act beyond natural language processing
- Create and run an enhanced **MCP server** that uses all three core capabilities:

- **Resources** → to fetch and expose data (read files, list directories)
- **Prompts** → to guide structured workflows (code review & documentation)
Additionally, your server will implement MCP Context to handle **logging, progress reporting, and user elicitation**
- Build a simple but extensible **MCP client** with a CLI menu to interact with your server and a [Claude](#) chatbot
- Understand how to extend MCP for your own projects

Workspace Setup

First let's run the following command to clone the project template from a Github repository:

 bash 

```
git clone -b start https://github.com/joshuazhou744/enhanced-mcp-server.git
```

 Run

This should create a directory `enhanced-mcp-server` where there will be empty `server.py` and `client.py` files along with a `requirements.txt` file that lists our required dependencies.

Before we install the dependencies, let's create a Python virtual environment named `.venv` to contain our development workspace.

 bash 

```
pip install virtualenv  
virtualenv .venv
```

 Run

Activate our virtual environment with:

 bash 

```
source .venv/bin/activate
```

 Run

Now that we are in our virtual environment, run the following command to change directories into the project root and install the necessary dependencies:

 bash 

```
cd enhanced-mcp-server  
pip install -r requirements.txt
```

 Run

The second command uses `pip`, a package manager for Python, with the `-r` flag, to recursively install all the dependencies in the file `requirements.txt`

Now that we have our workspace setup, let's start by building our server.

Server Part 1: Setup

Use the button below to open the `server.py` file, this is where we'll build our MCP server script.

Open server.py in IDE

At the top we'll import all our required dependencies.

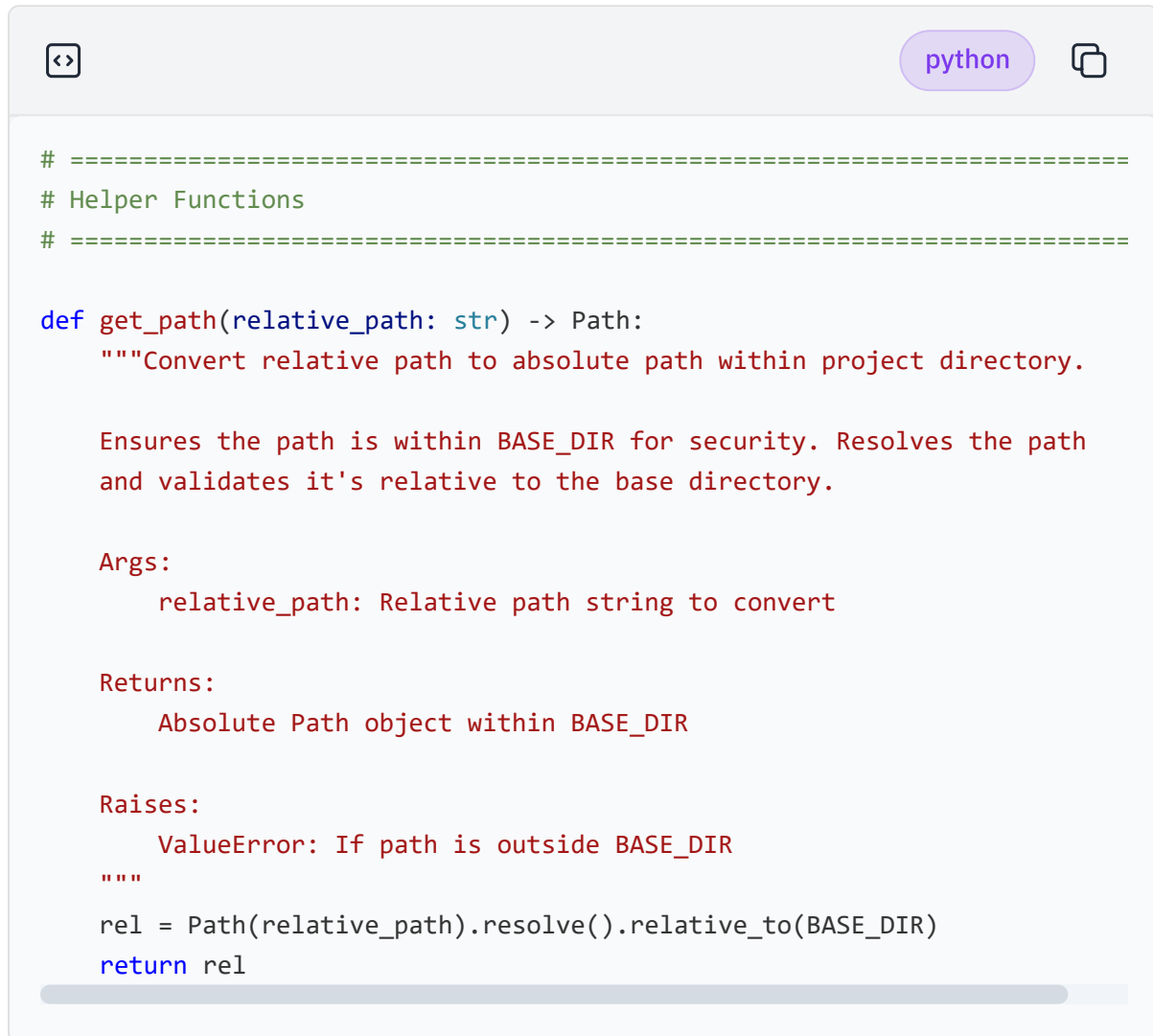
```
<> python   
  
from pathlib import Path  
from datetime import datetime  
from pydantic import BaseModel  
import time  
  
from fastmcp import FastMCP, Context
```

Now let's define a global variable, a schema, and our MCP server object.

```
<> python   
  
# Project root directory (current working directory)  
BASE_DIR = Path.cwd()  
  
class DocumentGeneratorSchema(BaseModel):  
    """Pydantic model for documentation filename schema.  
  
    Used in elicitation to capture user input for documentation file names  
  
    Attributes:  
        file_path: Path of the file we want to generate document on  
        name: The name of the documentation file to create  
    """  
    file_path: str  
    name: str  
  
# FastMCP server instance for handling MCP protocol operations  
mcp = FastMCP("File Operations MCP Server")
```

- Use `BASE_DIR` (the current absolute path) to standardize all our file paths to an absolute path so the file operations are and safe and reliable.
- Use `DocumentGeneratorSchema` to define the inputs needed for the document generation prompt you'll create later.

Finally, create a helper function that returns the absolute path (from `BASE_DIR`) when given a relative path.



```
# =====  
# Helper Functions  
# =====  
  
def get_path(relative_path: str) -> Path:  
    """Convert relative path to absolute path within project directory.  
  
    Ensures the path is within BASE_DIR for security. Resolves the path  
    and validates it's relative to the base directory.  
  
    Args:  
        relative_path: Relative path string to convert  
  
    Returns:  
        Absolute Path object within BASE_DIR  
  
    Raises:  
        ValueError: If path is outside BASE_DIR  
    """  
    rel = Path(relative_path).resolve().relative_to(BASE_DIR)  
    return rel
```

Now that we completed setup, we can move onto implementing the MCP server capabilities.

Server Part 2: Tools

MCP Context

When creating MCP servers, your tools, resources, and prompts may need access to the underlying MCP session or advanced MCP capabilities. The `Context` object from the **FastMCP** library (wrapper of the base MCP SDK, also has a `Context` module) provides a clean interface for accessing these features and capabilities.

You'll see these functionalities used in this project:

- **Logging:** Logs info & error messages
- **Progress Reporting:** Display progress of continuous tasks
- **User Elicitation:** Structured inputs from the user

Still in the `server.py` file.

Let's create two tools to interact with the files in the project directory.

The first tool is a function that creates a file given the file path (name) and contents.



python



```
# =====  
# TOOLS  
# =====  
  
@mcp.tool()  
async def write_file(file_path: str, content: str, ctx: Context) -> str:  
    """Create a new file with specified content.  
  
    Creates parent directories if they don't exist. Writes content to the  
    using UTF-8 encoding.  
  
    Args:  
        file_path: Relative path where the file should be created  
        content: Content to write to the file  
        ctx: MCP context for logging  
  
    Returns:  
        Success message with file path  
  
    Raises:  
        Exception: If file creation fails (logged to context)  
    """  
    try:  
        path = get_path(file_path)  
        # Create parent directories if they don't exist  
        path.parent.mkdir(parents=True, exist_ok=True)  
  
        total = len(content)  
        chunk_size = max(total // 10, 1)  
  
        # Write content with UTF-8 encoding with progress reporting  
        written = 0  
        with open(path, "w", encoding='utf-8') as f:  
            for i in range(0, total, chunk_size):  
                f.write(content[i:i+chunk_size])  
                written = min(i+chunk_size, total)  
                await ctx.report_progress(progress=written, total=total, r  
                    time.sleep(0.05)  
  
            await ctx.report_progress(progress=total, total=total, message="Wr  
            await ctx.info(f"File written successfully to: {file_path}")  
            return f"File written successfully to: {file_path}"  
    except Exception as e:  
        await ctx.error(f"Error creating file: {str(e)}")  
        raise
```


- First the function standardizes the relative path using the `get_path` helper function.
- Then it creates the parent directory if it doesn't exist, and writes the content into the file path.
 - The tool writes to the file in chunks and uses `Context` to report progress.
- Uses `Context` for logging and error messages.

The other tool deletes files rather than creating them.

```

@mcp.tool()
async def delete_file(file_path: str, ctx: Context) -> str:
    """Delete a file from the project directory.

    Validates that the path points to a file (not a directory) before delete

    Args:
        file_path: Relative path to the file to delete
        ctx: MCP context for logging

    Returns:
        Success or error message describing the operation result
    """
    try:
        path = get_path(file_path)
        # Check if path exists and is a file
        if path.is_file():
            path.unlink()
            await ctx.info(f"Successfully deleted file {file_path}")
            return f"Successfully deleted file {file_path}"
        elif path.is_dir():
            await ctx.warning(f"Error: {file_path} is a directory, not a file")
            return f"Error: {file_path} is a directory, not a file"
        else:
            await ctx.warning(f"File not found: {file_path}")
            return f"File not found: {file_path}"
    except Exception as e:
        await ctx.error(f"Error deleting file: {str(e)}")
        return f"Error deleting file: {str(e)}"

```

- Like `write_file`, it standardizes the relative path to an absolute path before performing any actions.

Then it checks if the path is a file and deletes it if not (i.e. a directory), we raise

- Again, **Context** is used for logging and error messages

That's it for tools, let's move onto **resources**.

Server Part 3: Resources

Still in the `server.py` file.

Uniform Resource Identifier (URI)

Resources are used to fetch and expose **typically static**, sometimes dynamic, information. This can be from a database, remote repository, or files local to the MCP server itself. If the *fetch* process is the main purpose of the resource, which is true more often than not, you are better off using a tool. For example, if you are fetching from remote repositories that requires special configuration, just make a tool with an input for the information we want.

MCP resources have a unique **Uniform Resource Identifier (URI)** system. In the header when defining an MCP resource, you define the path to the resource wanted, which can be parameterized. This allows for structured and organized resource identification. The static portion of URIs can be anything (for example, `dir://`, `docs://`), and are chosen by MCP server developers. This makes resources "hard-coded" in the sense that clients would need to know the specific URIs associated with their wanted resource.

All in all, resources are best for **passive data access** of readily available information whereas tools are better for **active operations**.

Resource Template

Resources that have parameterized URIs are known as resource templates. They serve the same function as normal resources but function a bit differently (dynamic URI). They also have a different method for being called in the client, we'll see this later.

Below, you'll create a resource template for fetching a file relative to the current directory.



python



```
# =====  
# RESOURCES  
# =====  
  
@mcp.resource("file:/// {file_name}")  
async def read_file_resource(file_name: str) -> dict:  
    """Read the content of a file as an MCP resource.  
  
    Provides file content access through the MCP resource protocol using  
    the file:/// URI scheme.  
  
    Args:  
        file_name: Relative path to the file to read  
  
    Returns:  
        Dictionary containing either file_content or error message  
    """  
    try:  
        path = get_path(file_name)  
  
        # Validate path exists and is a file  
        if not path.exists() or not path.is_file():  
            return {"error": f"Error: {file_name} is not a valid file"}  
        # Read and return file content  
        return {"file_content": path.read_text(encoding='utf-8')}  
  
    except Exception as e:  
        return {"error": f"Error reading file: {str(e)}"}
```

- Like before, start with getting the standardized path
- Then check if the path is a valid, existing file and return its contents
- There is a caveat to mention here:
 - The URI `file:///` has three forward slashes because `file://` is a reserved URI that expects a **host**, our host is the local directory so we put an extra `/`.

For the other resource we'll return a list of the files and directories in the current directory.



python



```
@mcp.resource("dir://.")
async def list_files_resource() -> dict:
    """List files and directories in the current directory as an MCP resou

    Provides directory listing through the MCP resource protocol using
    the dir:// URI scheme. Returns metadata for each item including name,
    path, type, size, and timestamps.

    Returns:
        Dictionary containing list of items with metadata, or error messag
    """
    try:
        path = get_path(".")
        if not path.exists() or not path.is_dir():
            return {"error": f"{path} is not a valid directory"}

        # Collect directory items with metadata
        items = []
        for item in path.iterdir():
            stat = item.stat()
            items.append({
                "name": item.name,
                "path": str(item.relative_to(BASE_DIR)),
                "type": "directory" if item.is_dir() else "file",
                "size": stat.st_size,
                "modified": datetime.fromtimestamp(stat.st_mtime).isoforma
                "created": datetime.fromtimestamp(stat.st_ctime).isoformat
            })

        return {
            "items": items
        }
    except Exception as e:
        return {"error": f"Error listing files: {e}"}
```

- The path is a simple `.` (current directory).
- A list of items is created and returned in a JSON object in the `items` field.

Now that we have two resources setup, we'll move onto **prompts**.

Server Part 4: Prompts

Still in the `server.py` file.

Prompts are structured templates that return a prompt message with the given parameters filled out. Prompts are designed to be **user-controlled** meaning users can choose a prompt based on their situation and fill out the parameters accordingly.

Let's create a prompt to perform a code review on a given file.



python



```
# =====
# PROMPTS
# =====

@mcp.prompt()
async def code_review(file_path: str, ctx: Context) -> str:
    """Generate a prompt for code review and quality evaluation.

    Reads a code file and generates a prompt for Claude to perform a
    comprehensive code review.

    Args:
        file_path: Relative path to the code file to review
        ctx: MCP context for logging and communication

    Returns:
        Formatted prompt string for code review

    Raises:
        FileNotFoundError: If the specified file doesn't exist
    """
    try:
        path = get_path(file_path)

        # Validate file exists
        if not path.exists() or not path.is_file():
            error_msg = f"Error: {file_path} is not a valid file"
            await ctx.warning(error_msg)
            raise FileNotFoundError(error_msg)

        # Read code and detect language from extension
        current_code = path.read_text(encoding='utf-8').strip()
        language = path.suffix.lower()

        # Generate structured prompt for code review
        prompt = f"""You are an expert code editor. Review the following c

File: {file_path}
Language (file suffix): {language or "unknown"}

Current code:
...
{current_code}
...

Provide a comprehensive evaluation of the code:

""".strip()
```

```
except Exception as e:
    await ctx.error(f"Error preparing code review prompt: {e}")
    raise
```

- First it checks that the target file for code review exists, this logic should not be handled inside the prompt but it's fine in this case.
- The content and language of the file are extracted and placed inside a long message, which is returned as the final prompt

Prompt templates can be filled out via simple function inputs that can be handled by the client (like above), or **user elicitation**.

User Elicitation

This feature using MCP Context allows servers to request structured inputs from users during prompt building or tool execution. It required a special handler function in the client which we will build later. It can be used in a wide range of scenarios:

- **Missing parameters:** Get unknown information that is required
- **Clarification requests:** Get user confirmation for ambiguous situations
- **Progressive disclosure:** Collect information gradually

Using elicitation, create a prompt to generate a separate documentation file of a given file.



python



```
@mcp.prompt()
async def documentation_generator(ctx: Context) -> str:
    """Generate a prompt for creating code documentation.

    Reads a code file, elicits a documentation filename from the user,
    and generates a prompt for Claude to create comprehensive documentation.

    Args:
        file_path: Relative path to the code file to document
        ctx: MCP context for logging and elicitation

    Returns:
        Formatted prompt string for documentation generation

    Raises:
        FileNotFoundError: If the specified file doesn't exist
    """
    try:
        # Elicit documentation filename from user via client
        result = await ctx.elicit(
            message="Please provide the subject file name and the document
            response_type=DocumentGeneratorSchema
        )

        file_path = result.data.file_path
        path = get_path(file_path)

        # Validate file exists
        if not path.exists() or not path.is_file():
            error_msg = f"Error: {file_path} is not a valid file"
            await ctx.warning(error_msg)
            raise FileNotFoundError(error_msg)

        # Read code and detect language from extension
        code = path.read_text(encoding='utf-8').strip()
        language = path.suffix.lower()

        doc_name = result.data.name

        # Generate structured prompt for documentation creation
        prompt = f"""You are an expert technical writer and documentation

File: {file_path}
Language (file suffix): {language or "unknown"}

Current code:
'''
{code}
'''
"""
```

- Use MCP tools available to you to create the separate documentation file:
- ****CRITICAL DETAIL: Name that separate document EXACTLY: {doc_name}****
 - Add the .md suffix yourself if the name doesn't include it already""".st

```
await ctx.info("Successfully returned prompt")
return prompt

except Exception as e:
    await ctx.error(f"Error generating code documentation prompt: {e}")
    raise
```

- Rather than using the function inputs, this prompt uses the `elicit` method from `Context` to get both the file to generate documentation on, and the new documentation file name.
- Again, there are checks that the file exists before extracting its content and language.
- Finally, the prompt is built with the extracted results and elicited new file name.

Before we finish `server.py`, we have to create an entry point that starts the MCP server when the script is executed.



```
# =====
# SERVER ENTRY POINT
# =====

if __name__ == "__main__":
    print("Starting File Operations Server...")
    mcp.run()
```

This ensures that the server runs and listens for MCP client connections via **standard input/output (STDIO)**.

Completed Server

Here is the completed code for `server.py`, make sure your version matches this version.



python



```
from pathlib import Path
from pydantic import BaseModel
from datetime import datetime
import time

from fastmcp import FastMCP, Context

# Project root directory (current working directory)
BASE_DIR = Path.cwd()

class DocumentGeneratorSchema(BaseModel):
    """Pydantic model for documentation filename schema.

    Used in elicitation to capture user input for documentation file names

    Attributes:
        file_path: Path of the file we want to generate document on
        name: The name of the documentation file to create
    """
    file_path: str
    name: str

# FastMCP server instance for handling MCP protocol operations
mcp = FastMCP("File Operations MCP Server")

# =====
# Helper Functions
# =====

def get_path(relative_path: str) -> Path:
    """Convert relative path to absolute path within project directory.

    Ensures the path is within BASE_DIR for security. Resolves the path
    and validates it's relative to the base directory.

    Args:
        relative_path: Relative path string to convert

    Returns:
        Absolute Path object within BASE_DIR

    Raises:
        ValueError: If path is outside base directory
    """
    try:
        rel = Path(relative_path).resolve().relative_to(BASE_DIR)
        return rel
    except ValueError:
```

```

# =====
# TOOLS
# =====

@mcp.tool()
async def write_file(file_path: str, content: str, ctx: Context) -> str:
    """Create a new file with specified content.

    Creates parent directories if they don't exist. Writes content to the
    using UTF-8 encoding.

    Args:
        file_path: Relative path where the file should be created
        content: Content to write to the file
        ctx: MCP context for logging

    Returns:
        Success message with file path

    Raises:
        Exception: If file creation fails (logged to context)
    """
    try:
        path = get_path(file_path)
        # Create parent directories if they don't exist
        path.parent.mkdir(parents=True, exist_ok=True)

        total = len(content)
        chunk_size = max(total // 10, 1)

        # Write content with UTF-8 encoding with progress reporting
        written = 0
        with open(path, "w", encoding='utf-8') as f:
            for i in range(0, total, chunk_size):
                f.write(content[i:i+chunk_size])
                written = min(i+chunk_size, total)
                await ctx.report_progress(progress=written, total=total, r
                    time.sleep(0.05)

            await ctx.report_progress(progress=total, total=total, message="Wr
            await ctx.info(f"File written successfully to: {file_path}")
            return f"File written successfully to: {file_path}"
    except Exception as e:
        await ctx.error(f"Error creating file: {str(e)}")
        raise

@mcp.tool()
async def delete_file(file_path: str, ctx: Context) -> str:
    """Delete a file from the project directory.

```

Args:

file_path: Relative path to the file to delete
ctx: MCP context for logging

Returns:

Success or error message describing the operation result

"""

try:

```
path = get_path(file_path)
# Check if path exists and is a file
if path.is_file():
    path.unlink()
    await ctx.info(f"Successfully deleted file {file_path}")
    return f"Successfully deleted file {file_path}"
elif path.is_dir():
    await ctx.warning(f"Error: {file_path} is a directory, not a f
    return f"Error: {file_path} is a directory, not a file"
else:
    await ctx.warning(f"File not found: {file_path}")
    return f"File not found: {file_path}"
except Exception as e:
    await ctx.error(f"Error deleting file: {str(e)}")
    return f"Error deleting file: {str(e)}"
```

```
# =====
# RESOURCES
# =====
```

```
@mcp.resource("file:///file_name")
```

```
async def read_file_resource(file_name: str) -> dict:
```

```
    """Read the content of a file as an MCP resource.
```

Provides file content access through the MCP resource protocol using
the file:/// URI scheme.

Args:

file_name: Relative path to the file to read

Returns:

Dictionary containing either file_content or error message

"""

try:

```
path = get_path(file_name)

# Validate path exists and is a file
if not path.exists() or not path.is_file():
    error_msg = f"Error: {file_name} is not a valid file"
    return {"error": error_msg}
# Read and return file content
return {"file_content": path.read_text(encoding='utf-8')}
```

```

        return {"error": f"Error reading file: {str(e)}"}

@mcp.resource("dir://.")
async def list_files_resource() -> dict:
    """List files and directories in the current directory as an MCP resou

    Provides directory listing through the MCP resource protocol using
    the dir:// URI scheme. Returns metadata for each item including name,
    path, type, size, and timestamps.

    Returns:
        Dictionary containing list of items with metadata, or error messag
    """
    try:
        path = get_path(".")
        if not path.exists() or not path.is_dir():
            return {"error": f"{path} is not a valid directory"}

        # Collect directory items with metadata
        items = []
        for item in path.iterdir():
            stat = item.stat()
            items.append({
                "name": item.name,
                "path": str(item.relative_to(BASE_DIR)),
                "type": "directory" if item.is_dir() else "file",
                "size": stat.st_size,
                "modified": datetime.fromtimestamp(stat.st_mtime).isoforma
                "created": datetime.fromtimestamp(stat.st_ctime).isoformat
            })

        return {
            "items": items
        }
    except Exception as e:
        return {"error": f"Error listing files: {e}"}

# =====
# PROMPTS
# =====

@mcp.prompt()
async def code_review(file_path: str, ctx: Context) -> str:
    """Generate a prompt for code review and quality evaluation.

    Reads a code file and generates a prompt for Claude to perform a
    comprehensive code review.

    Args:

```

Returns:

Formatted prompt string for code review

Raises:

FileNotFoundError: If the specified file doesn't exist

"""

try:

path = get_path(file_path)

Validate file exists

if not path.exists() or not path.is_file():

error_msg = f"Error: {file_path} is not a valid file"

await ctx.warning(error_msg)

raise FileNotFoundError(error_msg)

Read code and detect language from extension

current_code = path.read_text(encoding='utf-8').strip()

language = path.suffix.lower()

Generate structured prompt for code review

prompt = f"""You are an expert code editor. Review the following c

File: {file_path}

Language (file suffix): {language or "unknown"}

Current code:

...

{current_code}

...

Provide a comprehensive evaluation of the code:

""").strip()

await ctx.info("Successfully returned prompt")

return prompt

except Exception as e:

await ctx.error(f"Error preparing code review prompt: {e}")

raise

@mcp.prompt()

async def documentation_generator(ctx: Context) -> str:

"""Generate a prompt for creating code documentation.

Reads a code file, elicits a documentation filename from the user,
and generates a prompt for Claude to create comprehensive documentatio

Args:

file_path: Relative path to the code file to document

ctx: MCP context for logging and elicitation

Formatted prompt string for documentation generation

Raises:

FileNotFoundError: If the specified file doesn't exist

"""

try:

Elicit documentation filename from user via client

result = await ctx.elicit(

 message="Please provide the subject file name and the document

 response_type=DocumentGeneratorSchema

)

file_path = result.data.file_path

path = get_path(file_path)

Validate file exists

if not path.exists() or not path.is_file():

 error_msg = f"Error: {file_path} is not a valid file"

 await ctx.warning(error_msg)

 raise FileNotFoundError(error_msg)

Read code and detect language from extension

code = path.read_text(encoding='utf-8').strip()

language = path.suffix.lower()

doc_name = result.data.name

Generate structured prompt for documentation creation

prompt = f"""You are an expert technical writer and documentation

File: {file_path}

Language (file suffix): {language or "unknown"}

Current code:

...

{code}

...

Use MCP tools available to you to create the separate documentation file:

- **CRITICAL DETAIL: Name that separate document EXACTLY: {doc_name}**
- Add the .md suffix yourself if the name doesn't include it already""".st

await ctx.info("Successfully returned prompt")

return prompt

except Exception as e:

 await ctx.error(f"Error generating code documentation prompt: {e}")

 raise

=====

```
if __name__ == "__main__":  
    print("Starting File Operations Server...")  
    men.run()
```

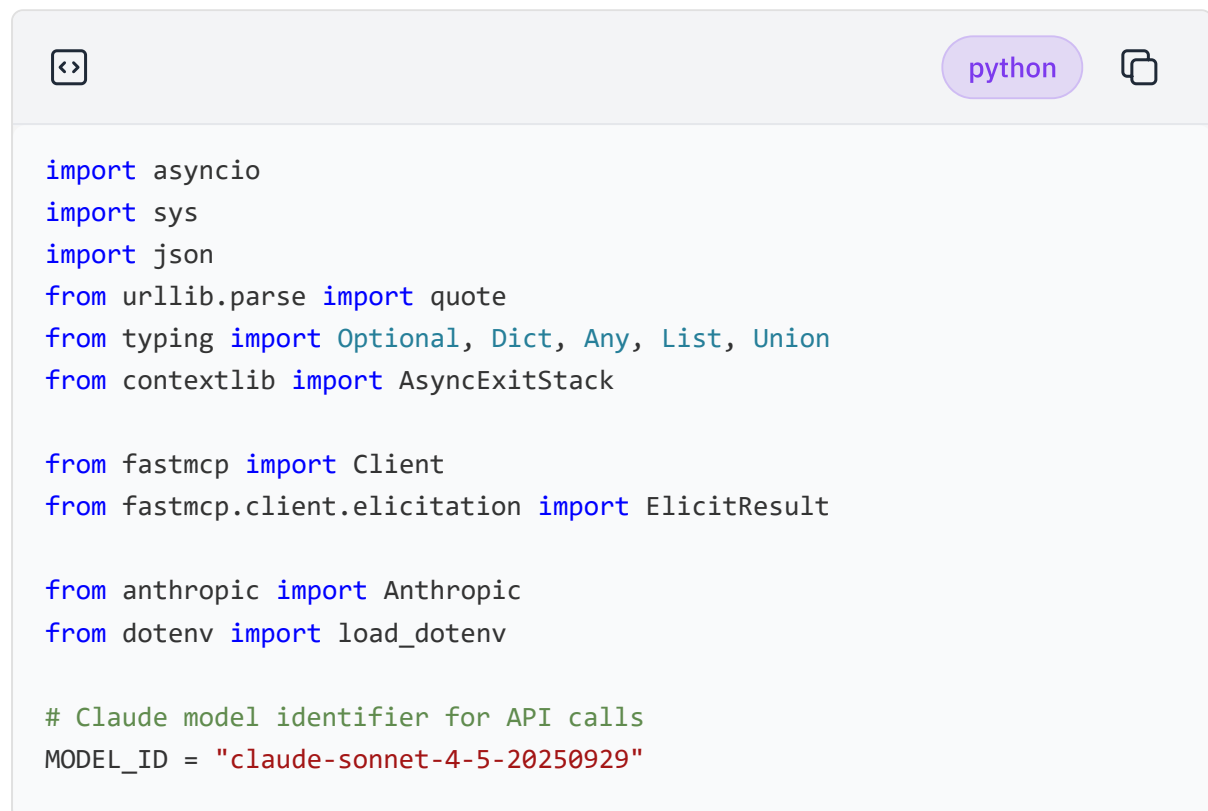
Now let's move on to building the **client**.

Client Part 1: Setup

Use the button below to open the `client.py` file, this is where we'll build our command line tool to interact with our MCP server.

Open client.py in IDE

At the top we'll import all required dependencies and define global variables.

A code editor window with a light gray header bar. On the left is a code icon, on the right is a 'python' language selector and a copy icon. The editor area contains Python code with syntax highlighting: imports for asyncio, sys, json, urllib.parse, typing, contextlib, fastmcp, anthropic, and dotenv; and a comment with a MODEL_ID variable.

```
import asyncio
import sys
import json
from urllib.parse import quote
from typing import Optional, Dict, Any, List, Union
from contextlib import AsyncExitStack

from fastmcp import Client
from fastmcp.client.elicitation import ElicitResult

from anthropic import Anthropic
from dotenv import load_dotenv

# Claude model identifier for API calls
MODEL_ID = "claude-sonnet-4-5-20250929"
```

API Disclaimer

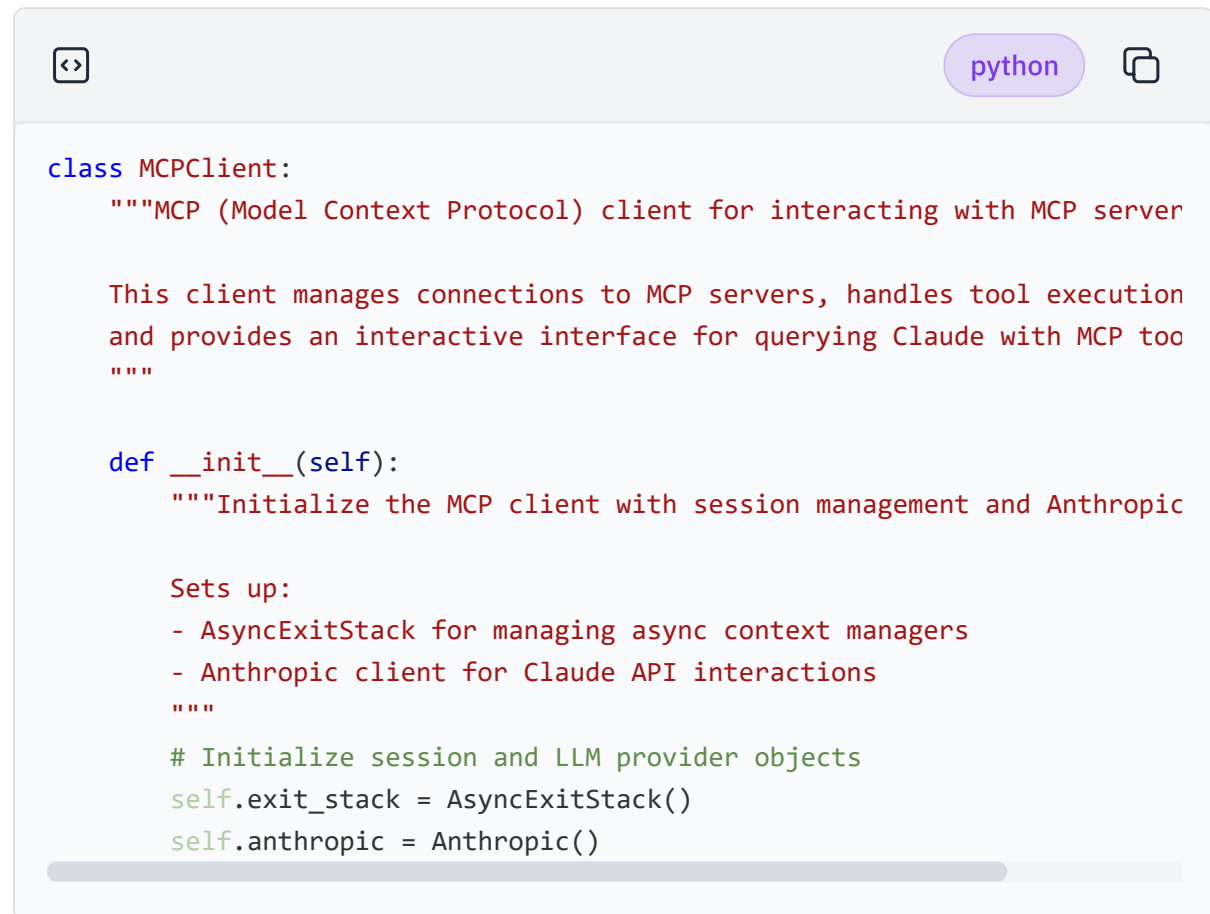
The client chatbot is powered by **Claude**, an LLM provided by **Anthropic** who are also the developers and maintainers of MCP. We'll use the Anthropic API to invoke the model defined by `MODEL_ID`, which requires a paid API key. However, in this CloudIDE lab environment, that API key is already configured for you so you don't need to worry about it.

Python Class

[Python classes](#) are object-oriented constructs that provide the perfect blueprint for our system because they can encapsulate data in attributes and provide functionality through methods in a single, organized instance.

Let's use a Python class for our MCP client.

From here up until the end of the client, everything will be inside this class, be mindful of the indentation.



```
class MCPClient:
    """MCP (Model Context Protocol) client for interacting with MCP server

    This client manages connections to MCP servers, handles tool execution
    and provides an interactive interface for querying Claude with MCP too
    """

    def __init__(self):
        """Initialize the MCP client with session management and Anthropic

        Sets up:
        - AsyncExitStack for managing async context managers
        - Anthropic client for Claude API interactions
        """

        # Initialize session and LLM provider objects
        self.exit_stack = AsyncExitStack()
        self.anthropic = Anthropic()
```

- The first method of our class is the `__init__` method that runs on initialization of every instance of `MCPClient`.
- On initialization, we'll define an exit stack to manage asynchronous context and the Anthropic API client for invoking Claude

Create a method to connect to the MCP server via **STDIO**, given the server script path.



python



```
async def connect_to_server(self, server_script_path: str):
    """Connect to an MCP server via stdio transport.

    Establishes a connection to an MCP server by launching the server
    as a subprocess and communicating via stdin/stdout.

    Args:
        server_script_path: Path to the server script (.py, .js, or .ts)

    Raises:
        ValueError: If server_script_path is not a .py, .js, or .ts file
    """
    # Determine script type based on file extension
    is_python = server_script_path.endswith('.py')
    is_ts = server_script_path.endswith('.ts')
    is_js = server_script_path.endswith('.js')

    if not (is_python or is_ts or is_js):
        raise ValueError("Server script must be a .py, .js, or .ts file")

    self.client = Client(
        server_script_path,
        elicitation_handler=self.handle_elicitation,
        progress_handler=self.handle_progress,
        message_handler=self.handle_message
    )

    await self.exit_stack.enter_async_context(self.client)
```

- First, there's a check for a valid MCP server file type (Python, JavaScript/TypeScript)
- Then a `Client` from the **FastMCP** library is instantiated and saved to a class attribute.
 - We have three handler functions passed in, that we will define below.
- Finally, we establish a live, asynchronous session between client and server using the `exit_stack` which also handles cleanup and closure.

Let's create the elicitation handler function.



python



```
async def handle_elicitation(self, message: str, response_type: type,
    """Handle elicitation requests from the MCP server.

    When the server needs user input, this handler prompts the user,
    collects their response, and returns it in the expected format.

    Args:
        message: The question or prompt from the server
        response_type: Pydantic model defining the expected response s
        params: Additional parameters for the elicitation
        context: Elicitation context information

    Returns:
        ElicitResult with action="decline" if no response, or response
    """
    print(f"Server asks: {message}")

    user_data = {}
    for field_name, field_type in response_type.__annotations__.items():
        user_input = input(f"Enter value for '{field_name}' ({field_ty
        if not user_input:
            return ElicitResult(action="decline")

        user_data[field_name] = user_input

    # Return the structured response object
    return response_type(**user_data)
```

- First the user is prompted with the `message` from the elicitation request from the server.
- Each field of the elicitation request is displayed to the user and expects an input, if no input we simply return an `ElicitResult` with an action of **decline**.
- All user inputs are stored in a JSON object and returned in the `response_type` schema that's given by the elicitation request.
- This function may seem a bit confusing, that's because a lot of the overhead is handled by the FastMCP library and this is simply the handler function format the `Client` object expects.

Next let's make a progress reporting handler.



python



```
async def handle_progress(self, progress: float, total: float | None,
    """Handle progress notifications from the MCP server.

    Displays progress updates to the user, showing percentage complete

    Args:
        progress: Current progress value
        total: Total expected progress value (None if unknown)
        message: Optional descriptive message about current progress
    """
    if total is not None:
        percentage = (progress / total) * 100
        print(f"Progress: {percentage:.1f}% - {message or ''}")
    else:
        print(f"Progress: {progress} - {message or ''}")
```

- This is a unidirectional process so it's much simpler.
- Simply take the progress and total and print it as a percentage.

Similarly, a message handler.



python



```
async def handle_message(self, message):
    """Handle notification messages from the MCP server.

    Processes server notifications such as tool list changes or resour
    and displays appropriate messages to the user.

    Args:
        message: MCP notification message from the server
    """
    if hasattr(message, 'root'):
        method = message.root.method
        print(f"Received: {method}")

        if method == "notifications/tools/list_changed":
            print("Tools have changed - might want to refresh tool cac
        elif method == "notifications/resources/list_changed":
            print("Resources have changed")
```

- All Context logs from the server will be printed by this handler function.

Now that we have the Client itself setup, we can configure its interactions with the server.

Client Part 2: Fetching from MCP server

Still in the `client.py` file.

To make things easier, we'll create **protected getter methods** that fetch the lists of tools, resources, and prompts for us. These methods will lead with a `_` to indicate they are protected or intended for use **within** the class.

First let's get the tools in a format understandable by Claude.

```
async def _get_tools(self) -> List[Dict[str, Any]]:
    """Retrieve available tools from the MCP server.

    Fetches the list of tools exposed by the server and formats them
    for use with the Claude API.

    Returns:
        List of tool definitions with name, description, and input sch
    """
    tools_response = await self.client.list_tools()
    # Format tools for Claude API compatibility
    tools = [
        {
            "name": tool.name,
            "description": tool.description or "MCP Tool",
            "input_schema": tool.inputSchema,
        }
        for tool in tools_response
    ]

    return tools
```

- Since Claude was made by the creators of MCP, the tools don't need special adapting or conversion.
 - We can simply `list_tools` from the client and create a list of tool objects and return that.

The others will require more complex configuration when calling individual prompts or resources. Hence, the getter functions return the original list of **prompts**, **resources**, and **resource templates**.

python

```
async def _get_prompts(self):
    """Retrieve available prompts from the MCP server.

    Returns:
        PromptsResponse containing available prompt templates
    """

    prompts_response = await self.client.list_prompts()
    return prompts_response
```

python

```
async def _get_resources(self):
    """Retrieve available resources from the MCP server.

    Returns:
        ResourcesResponse containing available resources
    """

    resources_response = await self.client.list_resources()
    return resources_response
```

python

```
async def _get_resource_templates(self):
    """Retrieve available resource templates from the MCP server.

    Returns:
        ResourceTemplatesResponse containing available resource templa
    """

    resource_templates_response = await self.client.list_resource_temp
    return resource_templates_response
```

Let's move on to processing queries and interacting with the LLM.

Client Part 3: LLM Interaction

Still in the `client.py` file.

This method, `process_query`, takes a user input and prompts an MCP tool enabled LLM to fulfill the user query. It's quite long so we'll break it into chunks, so be mindful of indentation.



```
async def process_query(self, query: str) -> str:
    """Process a query using Claude with access to MCP server tools.

    Implements an agentic loop where Claude can use MCP tools to answer
    the query. The loop continues until Claude provides a final response
    without requesting further tool use.

    Args:
        query: The user's query to process

    Returns:
        The final text response from Claude
    """
    # Initialize conversation with user query
    messages = [
        {
            "role": "user",
            "content": query
        }
    ]

    # Fetch available tools from MCP server
    available_tools = await self._get_tools()

    # Initial Claude API call with tools
    response = self.anthropic.messages.create(
        model=MODEL_ID,
        max_tokens=4096,
        messages=messages,
        tools=available_tools
    )
```

- First we initialize our list of messages starting with the user query

- Then we use the `_get_tools` method to get our list of Claude compatible tool objects.
- Finally we call the LLM with the tools passed in.

From here, the LLM response will either include or exclude tool usages. We need to create an agentic loop to check if the LLM chooses to call a tool and if so, invoke that tool and feed the response to the LLM.



python



```
# Agentic loop - continue while Claude requests tool use
while response.stop_reason == "tool_use":
    # Add Claude's response (including tool use requests) to convey
    messages.append({
        "role": "assistant",
        "content": response.content
    })

    # Execute all requested tool calls
    tool_results = []
    for content in response.content:
        if content.type == 'tool_use':
            tool_name = content.name
            tool_args = content.input

            try:
                # Call the tool via MCP session
                result = await self.client.call_tool(tool_name, tool_args)

                # Format result content
                if isinstance(result.content, list):
                    result_text = "\n".join([
                        c.text if hasattr(c, 'text') else str(c)
                        for c in result.content
                    ])
                else:
                    result_text = result.content

                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": content.id,
                    "content": result_text
                })

            except Exception as e:
                # Handle tool execution errors
                print(f"Error calling tool {tool_name}: {e}")
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": content.id,
                    "content": f"Error: {str(e)}",
                    "is_error": True
                })

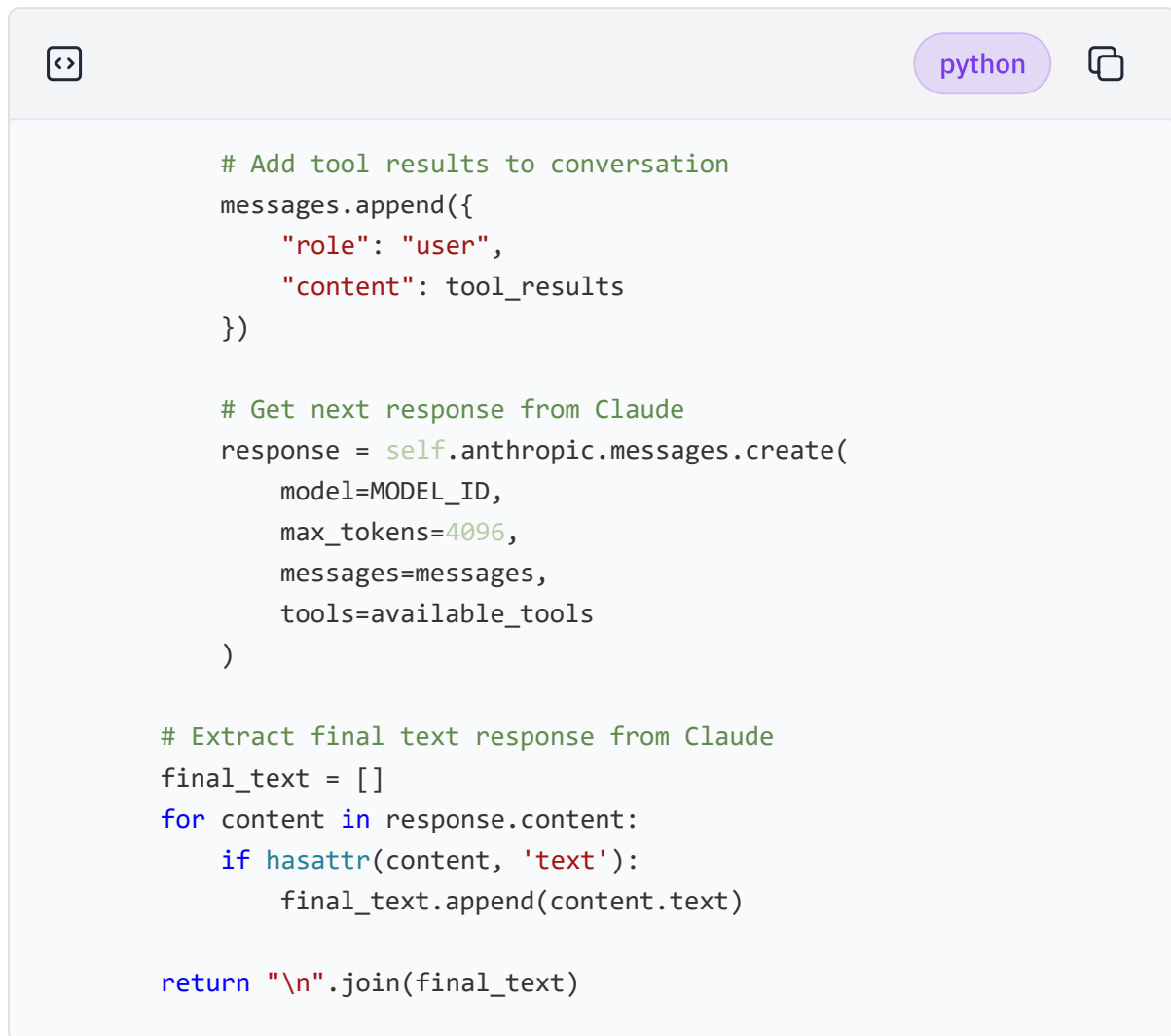
    # Append tool results to messages
    messages.append({
        "role": "system",
        "content": "Tool results: " + str(tool_results)
    })

    # Get next response from Claude
    response = await client.get_response(messages)
```

- The while loop essentially checks that the response includes a tool usage.
- The tool call content is appended to our list of messages

- This content contains the name of each tool and its arguments.
- Each tool call is processed if the type matches.
 - The tool name and arguments are extracted and used to call the tool.
 - Tool results are saved in a list `tool_results` as JSON objects with fields `type`, `tool_use_id`, `content`, and `is_error` (optional).

Finally we generate a final message after all tool calls are executed.



```

# Add tool results to conversation
messages.append({
    "role": "user",
    "content": tool_results
})

# Get next response from Claude
response = self.anthropic.messages.create(
    model=MODEL_ID,
    max_tokens=4096,
    messages=messages,
    tools=available_tools
)

# Extract final text response from Claude
final_text = []
for content in response.content:
    if hasattr(content, 'text'):
        final_text.append(content.text)

return "\n".join(final_text)

```

- Tool results are appended to `messages`.
- Claude is called with all the cumulative messages.
 - Initial user query
 - Tool call contents from the LLM
 - Tool call results from the MCP server
- The final text is extracted and formatted as a string for the final output seen by the user.

Before wrapping up this section, we'll create a helper function to `converse` with



python



```
async def converse(self):
    """Start an interactive conversation mode with Claude.

    Allows the user to have a multi-turn conversation with Claude,
    where each query can trigger tool use. Exits when user types 'quit'
    """
    print("\nEnter conversation mode. Type 'quit' or 'q' to exit.")

    while True:
        query = input("\nQuery: ").strip()

        if query.lower() in ("quit", "q"):
            break # Signal exit

        if not query:
            print("Please enter a query")
            continue

        try:
            response = await self.process_query(query)
            print("\n" + response)
        except Exception as e:
            print(f"Error processing query: {e}")

    return
```

- A simple loop that checks for "quit"/"q" inputs to exit the chat loop, calls `process_query` on all other inputs.

Now that our chatbot is built, let's move on to more specific workflows with **prompts**.

Client Part 4: Prompt Process

Still in the `client.py` file.

Workflows

Prompt templates are designed to be fed to an LLM to trigger a certain workflow. The structured inputs can be elicited from the user, or simply decided by an initial LLM call. As stated before, prompts are ultimately **user-controlled** "workflow triggers".

Let's create a method in our `MCPClient` to utilize prompts. This will be the sole method in this section so be aware of indentation.

```
async def prompt(self, prompt_name: str):
    """Execute a named prompt template from the MCP server.

    Retrieves a prompt template from the server, collects required arg
    from the user, generates the prompt, and processes it with Claude.

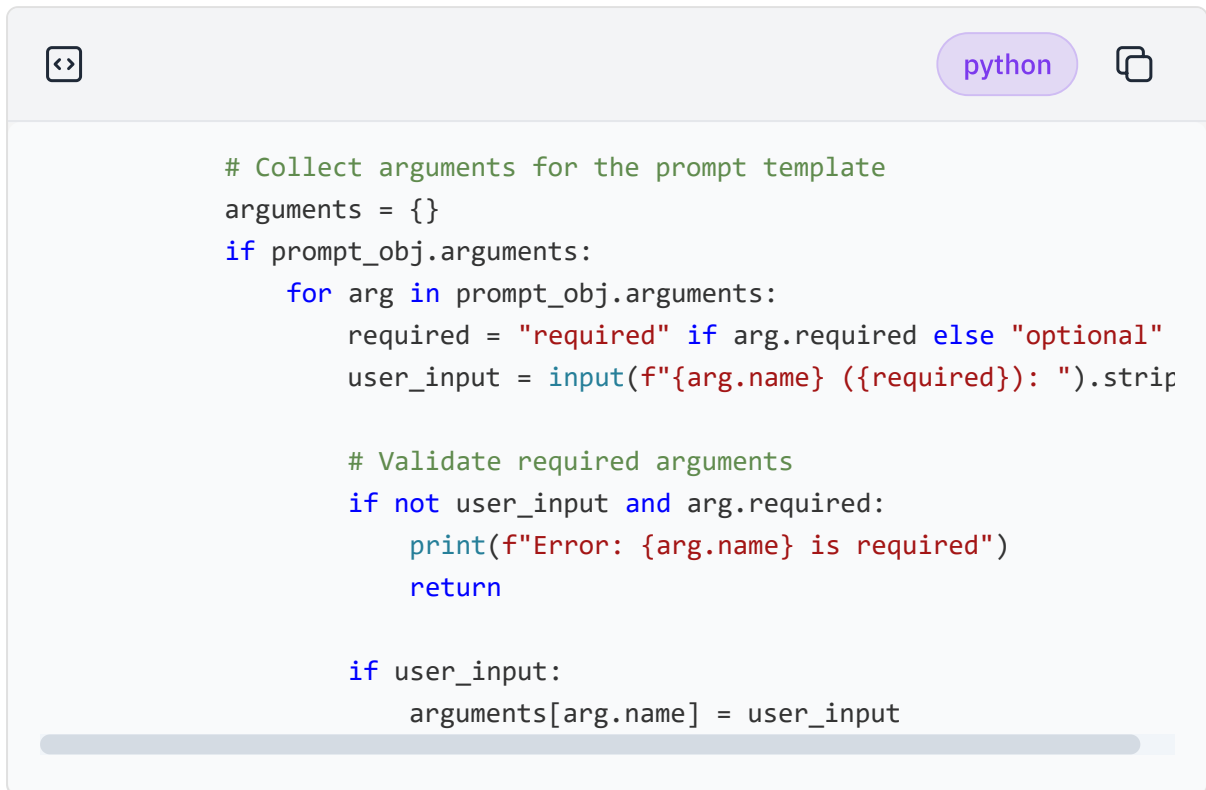
    Args:
        prompt_name: Name of the prompt template to execute
    """
    try:
        # Fetch available prompts from server
        prompts_response = await self._get_prompts()
        prompt_obj = next(
            (p for p in prompts_response if p.name == prompt_name),
            None
        )

        if not prompt_obj:
            print(f"Prompt '{prompt_name}' not found")
            return
```

- First we get the list of prompts.
- The we use the `next` function to iterate through the prompt that matches the

- The function builds a `prompt_obj` that has the arguments required for the prompt template.
- Finally, we perform a quick check that the `prompt_obj` exists.

Now let's prompt the user for those arguments.



```
# Collect arguments for the prompt template
arguments = {}
if prompt_obj.arguments:
    for arg in prompt_obj.arguments:
        required = "required" if arg.required else "optional"
        user_input = input(f"{arg.name} ({required}): ").strip()

        # Validate required arguments
        if not user_input and arg.required:
            print(f"Error: {arg.name} is required")
            return

        if user_input:
            arguments[arg.name] = user_input
```

- A map of empty arguments is instantiated.
- Iterate through each argument in the `prompt_obj`.
 - Get the user input for each argument, provide the label and whether or not it's **required**.
 - Save the argument to the map.

You may recall that one of our prompts, `code_review`, has arguments, whereas the other, `generate_documentation` gets its arguments through user elicitation. We did this to show different ways to get structured inputs in MCP.

Finally let's take our map, get the prompt template, and start the workflow.



python



```
# Generate the prompt with provided arguments
prompt_result = await self.client.get_prompt(prompt_name, argu

prompt = prompt_result.messages[0].content.text

# Process the generated prompt with Claude
response = await self.process_query(prompt)
print(response)
except Exception as e:
    print(f"Error: {type(e).__name__}: {e}\n")
    return
```

- Use the `get_prompt` method with parameters `prompt_name` and `arguments`.
- Get the text content of the prompt.
- Start the workflow and process the prompt as a query.
 - Print the response

Now we've completed the prompt process, let's move on to **resources**.

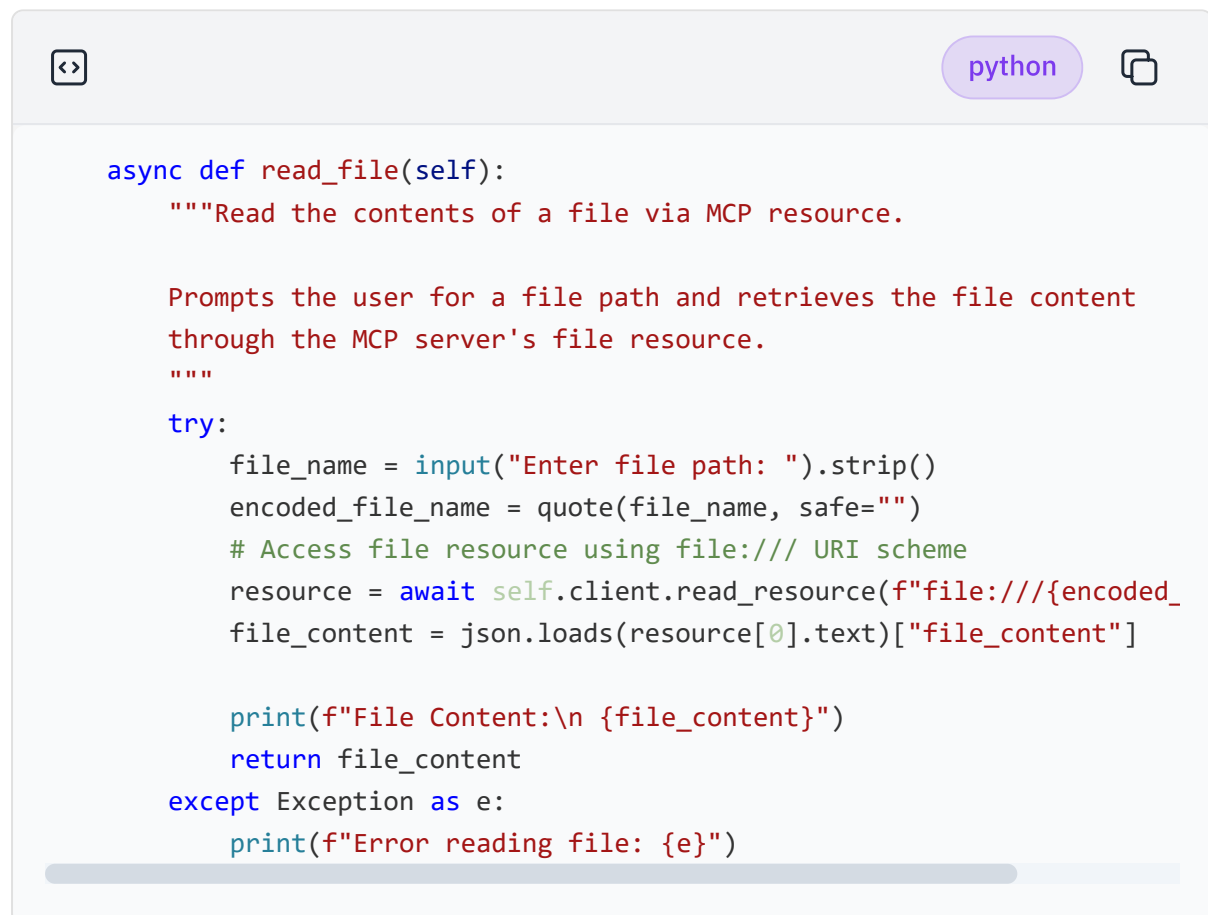
Client Part 5: Resources Reading

Still in the `client.py` file.

Resources and Resource Templates

You may recall the subset of resources: resource templates. Resource templates are parameterized resources that render dynamic resource dependent on the input parameters. There is no MCP mandate as to how the client invokes resources. That means the user could explicitly select or search for a resource. Additionally, an LLM could do the same thing at the request of a user. The user interaction model is very flexible.

For our application, we'll create a method for each resource in the MCP server, the first being the resource template with a user input parameter.



```
async def read_file(self):
    """Read the contents of a file via MCP resource.

    Prompts the user for a file path and retrieves the file content
    through the MCP server's file resource.
    """
    try:
        file_name = input("Enter file path: ").strip()
        encoded_file_name = quote(file_name, safe='')
        # Access file resource using file:/// URI scheme
        resource = await self.client.read_resource(f"file:/// {encoded_file_name}")
        file_content = json.loads(resource[0].text)["file_content"]

        print(f"File Content:\n {file_content}")
        return file_content
    except Exception as e:
        print(f"Error reading file: {e}")
```

- Get the `file_name` to read from the user and encode it as a URL for safe pathing.

- Parse the JSON response for the file content and print it out.

Before creating the next resource method, let's create a protected helper function to print out the listed directory.

```
def _print_dir_listing(self, items: list[dict]):  
    """Format and print a directory listing.  
  
    Args:  
        items: List of directory items with metadata (type, size, modi  
    """  
    print("\nDirectory Listing:\n")  
    print(f"{'Type':<10} {'Size':>10} {'Modified':<25} {'Name'}")  
    print("-" * 70)  
    for item in items:  
        # Add icon based on item type  
        type_icon = "📁" if item["type"] == "directory" else "📄"  
        size = f"{item['size']} B"  
        print(f"{type_icon:<2} {item['type']:<8} {size:>10} {item['mc
```

Let's create a method to read the contents of the current directory, this method calls a **static** resource.

```
async def read_dir(self):  
    """List the contents of the current directory via MCP resource.  
  
    Retrieves and displays directory contents through the MCP server's  
    directory resource.  
    """  
    try:  
        # Access directory resource using dir:// URI scheme  
        resource = await self.client.read_resource(f"dir://.")  
        dir_list = json.loads(resource[0].text)["items"]  
        self._print_dir_listing(dir_list)  
        return  
    except Exception as e:  
        print(f"Error reading directory: {e}")
```

- The `read_resource` method doesn't contain a parameter so the output remains constant throughout

Let's wrap up the application with a **CLI menu** to interact with each method we made in the client.

Client Part 6: CLI Menu and Cleanup

Still in the `client.py` file.

First let's create a method to display a menu of options each linked to an existing method that performs a specific action.



python



```
async def menu(self):
    """Run the main interactive chat loop with menu-driven interface.

    Presents a menu of options including prompt execution, file operat
    and conversation mode. Continues until user selects quit.
    """

    print("\nMCP Client Started!")
    print("Select from the menu or 'quit'/'q' to exit.")

    # Map menu choices to async functions
    menu_actions = {
        "1": lambda: self.prompt("documentation_generator"),
        "2": lambda: self.prompt("code_review"),
        "3": self.read_file,
        "4": self.read_dir,
        "5": self.converse,
        "q": self.quit_action,
        "quit": self.quit_action
    }

    while True:
        choice = input("""
Select from the Menu
1. Generate Documentation
2. Review Code
3. Read File
4. Read Current Directory
5. Converse with Agent
q. Quit
> """).strip()

        action = menu_actions.get(choice)

        if not action:
            print("Invalid choice. Please try again.")
            continue

        result = await action()
        if result == "quit":
            break
```

- The `menu_actions` maps each selection to the corresponding method.
- The loop prompts the user for their action choice and executes the corresponding action.

- If the action returns "quit" (`quit_action`), the loop breaks and the application shuts down.

Let's create the `quit_action` and `cleanup` methods to gracefully shut down the CLI menu.

```
<> python   
  
async def quit_action(self):  
    """Signal to exit the client.  
  
    Returns:  
        String "quit" to signal exit from chat loop  
    """  
    print("Exiting client...")  
    return "quit"  
  
async def cleanup(self):  
    """Clean up resources and close connections.  
  
    Closes the async exit stack which manages all open connections  
    and resources.  
    """  
    if self.exit_stack:  
        await self.exit_stack.aclose()
```

- Simply print "Exiting client" and return "quit".
- The `cleanup` method asynchronously closes the exit stack, shutting down the FastMCP `Client` gracefully.
 - Stops and closes all STDIO subprocesses and background tasks.

Outside the `MCPClient`, let's create a `main` function to run the `MCPClient` and its CLI menu.



python



```
async def main():
    # Check correct usage
    if len(sys.argv) < 2:
        print("Usage: python client.py <server_path>")
        sys.exit(1)

    client = MCPCClient()
    try:
        server_path = sys.argv[1]
        print(f"Connecting to server: {server_path}")

        await client.connect_to_server(server_path)

        await client.menu()
    except Exception as e:
        print(f"Error: {type(e).__name__}: {e}")
    finally:
        # Always cleanup at the end
        await client.cleanup()
```

- Checks that the script was properly run by `python client.py <server_path>`.
- Instantiates the `MCPCClient` class and connects it to the MCP server, then runs the `menu()` method.
- Runs the `cleanup` method after everything.

The main entrypoint for a python script, make sure to run the **asynchronous** `main` function in `asyncio`.



python



```
if __name__ == "__main__":
    asyncio.run(main())
```

Complete Client

Here is the code for the final `client.py` file, ensure it matches your version.



python



```
import asyncio
import sys
import json
from urllib.parse import quote
from typing import Optional, Dict, Any, List, Union
from contextlib import AsyncExitStack

from fastmcp import Client
from fastmcp.client.elicitation import ElicitResult

from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv() # Load environment variables from .env file

# Claude model identifier for API calls
MODEL_ID = "claude-sonnet-4-5-20250929"

class MCPClient:
    """MCP (Model Context Protocol) client for interacting with MCP server

    This client manages connections to MCP servers, handles tool execution
    and provides an interactive interface for querying Claude with MCP too
    """

    def __init__(self):
        """Initialize the MCP client with session management and Anthropic

        Sets up:
        - AsyncExitStack for managing async context managers
        - Anthropic client for Claude API interactions
        """
        # Initialize session and LLM provider objects
        self.exit_stack = AsyncExitStack()
        self.anthropic = Anthropic()

    async def connect_to_server(self, server_script_path: str):
        """Connect to an MCP server via stdio transport.

        Establishes a connection to an MCP server by launching the server
        as a subprocess and communicating via stdin/stdout.

        Args:
            server_script_path: Path to the server script (.py, .js, or .t

        Raises:
            ValueError: If server_script_path is not a .py, .js, or .ts fi
        """
```

```

is_ts = server_script_path.endswith('.ts')
is_js = server_script_path.endswith('.js')

if not (is_python or is_ts or is_js):
    raise ValueError("Server script must be a .py, .js, or .ts fil

self.client = Client(
    server_script_path,
    elicitation_handler=self.handle_elicitation,
    progress_handler=self.handle_progress,
    message_handler=self.handle_message
)

await self.exit_stack.enter_async_context(self.client)

async def handle_elicitation(self, message: str, response_type: type,
    """Handle elicitation requests from the MCP server.

When the server needs user input, this handler prompts the user,
collects their response, and returns it in the expected format.

Args:
    message: The question or prompt from the server
    response_type: Pydantic model defining the expected response s
    params: Additional parameters for the elicitation
    context: Elicitation context information

Returns:
    ElicitResult with action="decline" if no response, or response
    """
    print(f"Server asks: {message}")

    user_data = {}
    for field_name, field_type in response_type.__annotations__.items(
        user_input = input(f"Enter value for '{field_name}' ({field_ty
        if not user_input:
            return ElicitResult(action="decline")

        user_data[field_name] = user_input

    # Return the structured response object
    return response_type(**user_data)

async def handle_progress(self, progress: float, total: float | None,
    """Handle progress notifications from the MCP server.

Displays progress updates to the user, showing percentage complete

Args:
    progress: Current progress value
    total: Total expected progress value (None if unknown)

```

```

if total is not None:
    percentage = (progress / total) * 100
    print(f"Progress: {percentage:.1f}% - {message or ''}")
else:
    print(f"Progress: {progress} - {message or ''}")

async def handle_message(self, message):
    """Handle notification messages from the MCP server.

    Processes server notifications such as tool list changes or resource
    and displays appropriate messages to the user.

    Args:
        message: MCP notification message from the server
    """
    if hasattr(message, 'root'):
        method = message.root.method
        print(f"Received: {method}")

        if method == "notifications/tools/list_changed":
            print("Tools have changed - might want to refresh tool cache")
        elif method == "notifications/resources/list_changed":
            print("Resources have changed")

async def _get_tools(self) -> List[Dict[str, Any]]:
    """Retrieve available tools from the MCP server.

    Fetches the list of tools exposed by the server and formats them
    for use with the Claude API.

    Returns:
        List of tool definitions with name, description, and input schema
    """
    tools_response = await self.client.list_tools()
    # Format tools for Claude API compatibility
    tools = [
        {
            "name": tool.name,
            "description": tool.description or "MCP Tool",
            "input_schema": tool.inputSchema,
        }
        for tool in tools_response
    ]

    return tools

async def _get_prompts(self):
    """Retrieve available prompts from the MCP server.

```

```

"""

prompts_response = await self.client.list_prompts()

return prompts_response

async def _get_resources(self):
    """Retrieve available resources from the MCP server.

    Returns:
        ResourcesResponse containing available resources
    """
    resources_response = await self.client.list_resources()
    return resources_response

async def _get_resource_templates(self):
    """Retrieve available resource templates from the MCP server.

    Returns:
        ResourceTemplatesResponse containing available resource templa
    """
    resource_templates_response = await self.client.list_resource_temp
    return resource_templates_response

async def process_query(self, query: str) -> str:
    """Process a query using Claude with access to MCP server tools.

    Implements an agentic loop where Claude can use MCP tools to answe
    the query. The loop continues until Claude provides a final respon
    without requesting further tool use.

    Args:
        query: The user's query to process

    Returns:
        The final text response from Claude
    """
    # Initialize conversation with user query
    messages = [
        {
            "role": "user",
            "content": query
        }
    ]

    # Fetch available tools from MCP server
    available_tools = await self._get_tools()

    # Initial Claude API call with tools
    response = self.anthropic.messages.create(
        model=MODEL_ID,

```

```

tools=available_tools
)

# Agentic loop - continue while Claude requests tool use
while response.stop_reason == "tool_use":
    # Add Claude's response (including tool use requests) to conversation
    messages.append({
        "role": "assistant",
        "content": response.content
    })

    # Execute all requested tool calls
    tool_results = []
    for content in response.content:
        if content.type == 'tool_use':
            tool_name = content.name
            tool_args = content.input

            try:
                # Call the tool via MCP session
                result = await self.client.call_tool(tool_name, tool_args)

                # Format result content
                if isinstance(result.content, list):
                    result_text = "\n".join([
                        c.text if hasattr(c, 'text') else str(c)
                        for c in result.content
                    ])
                else:
                    result_text = result.content

                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": content.id,
                    "content": result_text
                })

            except Exception as e:
                # Handle tool execution errors
                print(f"Error calling tool {tool_name}: {e}")
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": content.id,
                    "content": f"Error: {str(e)}",
                    "is_error": True
                })

    # Add tool results to conversation
    messages.append({
        "role": "user",
        "content": tool_results
    })

```

```

        # Get next response from Claude
        response = self.anthropic.messages.create(
            model=MODEL_ID,
            max_tokens=4096,
            messages=messages,
            tools=available_tools
        )

    # Extract final text response from Claude
    final_text = []
    for content in response.content:
        if hasattr(content, 'text'):
            final_text.append(content.text)

    return "\n".join(final_text)

async def converse(self):
    """Start an interactive conversation mode with Claude.

    Allows the user to have a multi-turn conversation with Claude,
    where each query can trigger tool use. Exits when user types 'quit'
    """
    print("\nEnter conversation mode. Type 'quit' or 'q' to exit.")

    while True:
        query = input("\nQuery: ").strip()

        if query.lower() in ("quit", "q"):
            break # Signal exit

        if not query:
            print("Please enter a query")
            continue

        try:
            response = await self.process_query(query)
            print("\n" + response)
        except Exception as e:
            print(f"Error processing query: {e}")

    return

async def prompt(self, prompt_name: str):
    """Execute a named prompt template from the MCP server.

    Retrieves a prompt template from the server, collects required arg
    from the user, generates the prompt, and processes it with Claude.

    Args:
        prompt_name: Name of the prompt template to execute
    """
    try:

```



```

prompt_obj = next(
    (p for p in prompts_response if p.name == prompt_name),
    None
)

if not prompt_obj:
    print(f"Prompt '{prompt_name}' not found")
    return

print(prompt_obj)

# Collect arguments for the prompt template
arguments = {}
if prompt_obj.arguments:
    for arg in prompt_obj.arguments:
        required = "required" if arg.required else "optional"
        user_input = input(f"{arg.name} ({required}): ").strip()

        # Validate required arguments
        if not user_input and arg.required:
            print(f"Error: {arg.name} is required")
            return

        if user_input:
            arguments[arg.name] = user_input

# Generate the prompt with provided arguments
prompt_result = await self.client.get_prompt(prompt_name, arguments)

prompt = prompt_result.messages[0].content.text

# Process the generated prompt with Claude
response = await self.process_query(prompt)
print(response)
except Exception as e:
    print(f"Error: {type(e).__name__}: {e}\n")
    return

async def read_file(self):
    """Read the contents of a file via MCP resource.

    Prompts the user for a file path and retrieves the file content
    through the MCP server's file resource.
    """
    try:
        file_name = input("Enter file path: ").strip()
        encoded_file_name = quote(file_name, safe='')
        # Access file resource using file:/// URI scheme
        resource = await self.client.read_resource(f"file://{encoded_file_name}")
        file_content = json.loads(resource[0].text)["file_content"]

```

```

        return file_content
    except Exception as e:
        print(f"Error reading file: {e}")

def _print_dir_listing(self, items: list[dict]):
    """Format and print a directory listing.

    Args:
        items: List of directory items with metadata (type, size, modi
    """
    print("\nDirectory Listing:\n")
    print(f"{'Type':<10} {'Size':>10} {'Modified':<25} {'Name'}")
    print("-" * 70)
    for item in items:
        # Add icon based on item type
        type_icon = "📁" if item["type"] == "directory" else "📄"
        size = f"{item['size']} B"
        print(f"{type_icon:<2} {item['type']:<8} {size:>10} {item['mc

async def read_dir(self):
    """List the contents of the current directory via MCP resource.

    Retrieves and displays directory contents through the MCP server's
    directory resource.
    """
    try:
        # Access directory resource using dir:// URI scheme
        resource = await self.client.read_resource(f"dir://.")
        dir_list = json.loads(resource[0].text)["items"]
        self._print_dir_listing(dir_list)
        return
    except Exception as e:
        print(f"Error reading directory: {e}")

async def menu(self):
    """Run the main interactive chat loop with menu-driven interface.

    Presents a menu of options including prompt execution, file operat
    and conversation mode. Continues until user selects quit.
    """
    print("\nMCP Client Started!")
    print("Select from the menu or 'quit'/'q' to exit.")

    # Map menu choices to async functions
    menu_actions = {
        "1": lambda: self.prompt("documentation_generator"),
        "2": lambda: self.prompt("code_review"),
        "3": self.read_file,
        "4": self.read_dir,
        "5": self.converse,

```

```

    }

    while True:
        choice = input("""
Select from the Menu
1. Generate Documentation
2. Review Code
3. Read File
4. Read Current Directory
5. Converse with Agent
q. Quit
> """).strip()

        action = menu_actions.get(choice)

        if not action:
            print("Invalid choice. Please try again.")
            continue

        result = await action()
        if result == "quit":
            break

    async def quit_action(self):
        """Signal to exit the client.

        Returns:
            String "quit" to signal exit from chat loop
        """
        print("Exiting client...")
        return "quit"

    async def cleanup(self):
        """Clean up resources and close connections.

        Closes the async exit stack which manages all open connections
        and resources.
        """
        if self.exit_stack:
            await self.exit_stack.aclose()

    async def main():
        # Check correct usage
        if len(sys.argv) < 2:
            print("Usage: python client.py <server_path>")
            sys.exit(1)

        client = MCPClient()
        try:
            server_path = sys.argv[1]
            print(f"Connecting to server: {server_path}")

```

```
        await client.menu()
    except Exception as e:
        print(f"Error: {type(e).__name__}: {e}")
    finally:
        # Always cleanup at the end
        await client.cleanup()

if __name__ == "__main__":
    asyncio.run(main())
```

All done! Now it's time to run the application.

Run the Application

Navigate to the **Terminal** and make sure you are in the `enhanced-mcp-server` directory.

Run the command:

 bash 

```
python client.py server.py
```



The output should look like this:

 plaintext 

```
MCP Client Started!  
Select from the menu or 'quit'/'q' to exit.  
  
Select from the Menu  
1. Generate Documentation  
2. Review Code  
3. Read File  
4. Read Current Directory  
5. Converse with Agent  
q. Quit
```

Here is your task:

- Explore the application.
- Try out all the features.
- Try to break the application.
- Find and resolve any bugs you can find.

(You can start by conversing with the Agent and asking it to help create code for

Be mindful when conversing with the LLM, you have a limited number of API calls.

What to Observe

As you test each option in the menu, pay attention to how the client and server interact:

- The **progress updates** you see come from `ctx.report_progress()` in your tools.
- When you generate documentation, notice how the **elicitation** prompt asks for inputs.
- Resource calls (`file:///` or `dir://.`) show how **URIs** map to real file operations.
- Watch the **logs** printed by the server — they show how the MCP Context tracks actions and messages.

Troubleshooting Tips

If something doesn't work as expected:

- Ensure your **virtual environment** is activated (`source .venv/bin/activate`).
- Make sure both `server.py` and `client.py` are saved in the **same directory**.
- If you see a `"ValueError: Server script must be a .py, .js, or .ts file"`, check that you're passing `server.py` correctly.
- Restart the server after editing it, the client connects to a running process that doesn't auto update.

Conclusion

🎉 Congratulations! You've just built a complete **MCP server and command line client!**

Here's what you accomplished:

- Set up a Python project with **FastMCP**
- Implemented **tools** for file creation and deletion
- Exposed **resources** for reading files and directories
- Added **prompts** to guide workflows like code review and documentation
- Built an **interactive client** that talks to your server and leverages **Anthropic's Claude**

With this foundation, you now understand how the **Model Context Protocol** allows LLMs to interact with the outside world in a structured, extensible way.

Next Steps

Here are some directions to keep building:

- Add more **custom tools**, for example, file search, code formatting, or database access
- Create **new resources**, such as exposing an API or system metrics
- Expand your **prompts** into multi-step workflows for debugging, data analysis, or automation
- Explore how to secure and sandbox MCP servers for real-world deployment

Remember, MCP isn't just about files — it's a universal bridge between AI and its environment.

Author(s)

[Abdul Fatir | Data Scientist @ IBM](#) [Joshua Zhou | Data Scientist @ IBM](#) [Joseph Santarcangelo | Data Scientist @ IBM](#)

Build an Enhanced MCP Server

Estimated time needed: 30 minutes

Ever wished your AI assistant could not only talk, but also read, write, and interact directly with your files? With the Model Context Protocol, you can widen an LLM's capabilities through tools, resources and prompts.

Model Context Protocol

[Large Language Models](#) (LLMs) are powerful, but on their own, they can't do much outside of text generation. That's where the **Model Context Protocol (MCP)** comes in. MCP is a standardized way to let models talk to their environment using **tools, resources, and prompts**.

How MCP Differs from Normal APIs

- **Standardization & modularity** → MCP provides a universal protocol that any LLM can use directly, instead of bespoke API integrations.
- **Three capability types** → Unlike APIs that just expose endpoints, MCP formalizes **tools, resources, and prompts** to cover actions, data, and workflows.
- **Contextual & interactive** → MCP servers run alongside the model, enabling real-time, two-way interaction (including eliciting missing info), not just one-way request/response.

Learning Objectives

By completing this lab you will be able to:

- Explain what the **Model Context Protocol** is and why it matters for enabling LLMs to act beyond natural language processing
- Create and run an enhanced **MCP server** that uses all three core capabilities:
 - **Tools** → to perform actions (write & delete files)
 - **Resources** → to fetch and expose data (read files, list directories)
 - **Prompts** → to guide structured workflows (code review & documentation)Additionally, your server will implement MCP Context to handle **logging, progress reporting, and user elicitation**
- Build a simple but extensible **MCP client** with a CLI menu to interact with your server and a [Claude](#) chatbot
- Understand how to extend MCP for your own projects

Workspace Setup →